



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

中介者模式（Mediator）

一、意图（Intent）

在嵌入式 C 里，中介者模式的核心不是“多一个中心对象”，而是：

把多个模块之间本来杂乱的直接调用，收拢到一个统一协调中心，让各模块只向中心汇报状态、提交消息或请求协作，而不直接彼此耦合。

它控制的是：

- 模块之间的信息交换路径
- 协作规则由谁集中决定
- 哪些动作需要被串行化、转发或延后

所以中介者模式本质上是在控制**模块协作关系**。

二、本质 / 动机（Motivation）

嵌入式系统一开始往往只有几个模块：

- 传感器
- 通信
- UI
- 电源管理
- 存储

当模块变多后，如果它们彼此直接调用，很快就会形成一张网：

- 传感器直接通知 UI
- UI 直接控制蜂鸣器
- 通信模块直接改状态机
- 电源管理直接回调多个业务模块

短期看似方便，长期却很危险。

1. 依赖关系迅速膨胀

当一个模块知道很多别的模块时：

- 初始化顺序难安排
- 替换模块很麻烦
- 单元测试很难做
- 复用困难

2. 协作逻辑分散

真正复杂的往往不是“某个模块自己做什么”，而是“它和别人怎么配合”。
若协作逻辑分散在各个模块内部，系统会越来越不可读。

3. 上下文和时序越来越难控

直接互调时，经常会出现：

- 在 ISR 里直接碰 UI 或通信栈
- 低优先级模块直接触发高代价动作
- 多个模块抢着改同一份系统状态

4. 很多开源 RTOS 和厂家 SDK 默认采用事件循环 / 软件总线 / 中央协调器

也就是说，它们并不鼓励模块之间互相强绑定，而是让模块：

- 发布事件
- 注册处理函数
- 通过中心总线传递消息
- 由统一协调层完成联动

这正是中介者模式在嵌入式中的自然落点。

三、适用性（Applicability）

中介者模式适合这些条件：

- **对象或模块很多**：存在多对多交互趋势
- **交互规则比模块本身更复杂**

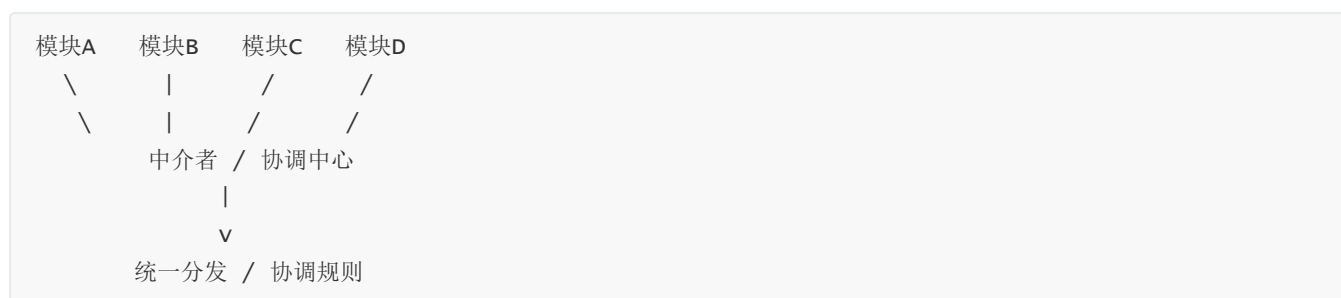
- **希望减少直接引用**：模块最好彼此不知道实现细节
- **需要统一调度上下文**：事件要在合适线程/循环中执行
- **协作规则经常变化**：流程会改，但基础模块相对稳定

一句话判断：

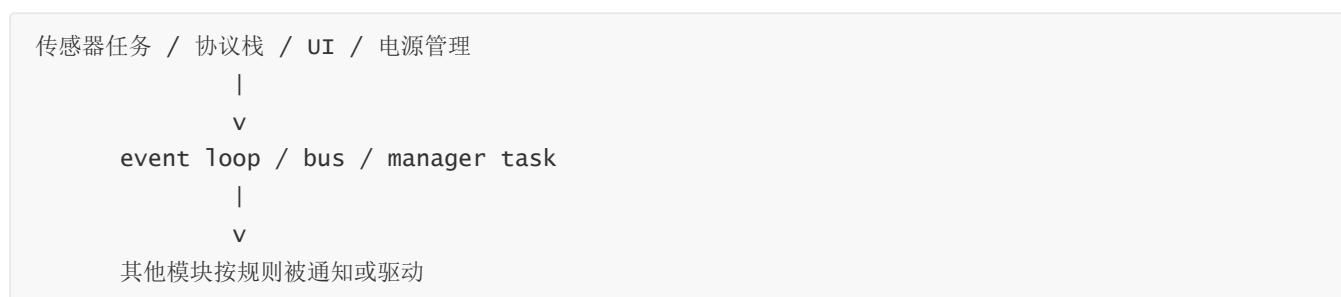
如果系统真正难维护的部分，是模块之间怎么配合，而不是单个模块怎么做事，中介者模式通常就很有价值。

四、结构（Structure）

从嵌入式 C 的角度，可把它看成：



更贴近工程落地时，常见结构是：



这里的关键转变是：

- 原来是模块与模块直接互调
- 现在是模块只与中介者打交道

网络状依赖被收敛成星型依赖。

五、参与者（Participants）

1. Mediator（中介者接口）

它定义统一协调方式。

在嵌入式里，可能体现为：

- 事件发布接口
- 消息投递接口
- 模块协作回调入口
- 统一调度器接口

2. ConcreteMediator（具体中介者）

它是真正的协调中心，负责：

- 保存观察者 / 订阅者关系
- 根据事件决定要通知谁
- 在需要时串行化执行
- 维持某些全局协作状态

它不是简单转发器，而是交互规则真正集中的地方。

3. Colleague（同事模块）

这些模块各自完成自己的职责：

- 采样
- 通信
- 显示
- 存储
- 告警

但它们不直接知道其他模块的细节。

它们只知道如何向中介者上报或订阅。

4. Message / Event / Context（消息与上下文）

嵌入式里的中介者通常离不开消息模型。

典型内容包括：

- 事件类型
- 数据载荷
- 来源模块
- 时间戳
- 状态标志

中介者是否清晰，通常取决于这套消息模型是否清楚。

5. Client（装配者）

Client 负责创建中介者、注册模块、建立观察关系。

在产品代码里，它通常就是系统初始化阶段。

六、协作（Collaborations）

中介者模式的典型执行路径是：

1. 某个模块产生状态变化或事件
2. 该模块不直接调用其他模块，而是通知中介者

3. 中介者根据规则决定：

- 通知谁
- 以什么顺序通知
- 是否同步还是异步
- 是否需要切换上下文

4. 被影响的模块再执行各自动作

因此真正复杂的部分，不再散落在多个模块内部，而是集中在中介者的协作规则中。

这会让系统更容易看懂：

- 模块自己负责什么
- 协调中心负责什么
- 哪些联动是系统级规则

七、效果（Consequences）

收益

1. 降低模块间直接耦合

模块不再互相持有引用，也不需要知道彼此内部实现。

2. 协作规则集中管理

系统联动、跨模块流程、统一调度规则都能在一个地方收敛。

3. 更利于替换与裁剪

把某个模块换成另一实现时，只要它仍遵守中介者协议，其他模块通常不必大改。

4. 更适合控制执行上下文

通过 event loop、bus 或 manager task，可以把跨模块动作拉到受控上下文里，减少乱序调用。

代价

1. 中介者可能膨胀

如果所有逻辑都堆进去，中介者会退化成“上帝模块”。

2. 复杂性只是被集中，不是消失

系统协作仍然复杂，只是从分散复杂改成集中复杂。

3. 简单系统未必值得引入

如果模块很少、关系很简单，直接调用反而更轻。

八、实现要点 (Implementation)

1. 先设计消息边界

中介者不应该转发一堆零散全局变量，而应该有清晰的事件或消息定义。

2. 明确同步与异步策略

有些通知可以同步调用，有些必须排队延后。
这在实时系统里非常关键。

3. 不要让同事模块绕过中介者

如果模块一边通过总线协作，一边又偷偷彼此直调，那么耦合问题会重新长回来。

4. 中介者只负责协作，不抢业务主体职责

它应该负责：

- 路由
- 协调
- 串行化
- 联动规则

而不应吞掉所有业务实现。

5. 注意消息所有权和生命周期

尤其在 C 里，要明确：

- 发布时是复制消息还是传引用
- 观察者读到的是快照还是共享区
- ISR 能否直接发布
- 消息在何时失效

九、典型应用

下面这些都是很多开源项目和厂家 SDK 默认会给出的组织方式：

1. ESP-IDF Event Loop

ESP-IDF 的事件循环允许组件声明事件、注册处理器，并把处理动作放到事件循环这一桥梁上完成。
它强调的是松耦合组件协作，而不是模块之间互相直调。

2. Zephyr zbus

Zephyr zbus 通过 channel 与 observer 组织线程、回调和消息传递，让模块通过软件总线协作。这是非常典型的中介者式架构。

3. 无线/网络 SDK 的中央事件分发

很多 Wi-Fi、BLE、蜂窝模组 SDK 都默认提供 central event handler、event loop 或 manager task。原因很简单：连接状态、IP 状态、扫描结果、重连动作这些联动如果分散到各模块互调，会很快失控。

4. 应用层中心状态管理器

在大量量产产品里，传感器、告警、显示、蜂鸣器、上报模块不会彼此直接纠缠，而是由中心管理器根据事件统一协调。

这也是中介者模式在工程里非常常见的落点。

总结

中介者模式在嵌入式 C 里的本质，就是把模块之间的直接网状协作，收拢到一个统一协调中心，让模块只关心自己的职责，不直接彼此缠绕。